



DESARROLLO DE APLICACIONES CON REACT

Desarrollo Front-End

Carlos Rojas Sánchez

Ingeniería en Tecnologías Computacionales

Universidad del Mar

¿Qué es React?

- Es una **biblioteca** (no un framework) de JavaScript.
- Especializada en la creación de interfaces de usuario (UI).
- Mantenida por **Meta** (Facebook) y una comunidad masiva.
- Lanzada originalmente en 2013.

Enfoque

Se centra exclusivamente en la "Vista" (V) del patrón MVC o en la capa de interfaz.

El Paradigma Declarativo

React se basa en la programación declarativa:

- Tú describes **qué** quieres ver en pantalla (el estado final).
- React se encarga de **cómo** actualizar el DOM de forma eficiente.

Diferencia con el imperativo: No manipulamos el DOM manualmente (ej. no usamos `document.getElementById` para cambiar un texto).

React divide la interfaz en piezas independientes:

- **Encapsulación:** Cada componente tiene su propia lógica y estilo.
- **Reutilización:** Un botón, una barra de navegación o una tarjeta se escriben una vez y se usan en toda la app.
- **Composición:** Las aplicaciones se construyen uniendo pequeños componentes para formar estructuras complejas.

¿Por qué elegir React?

1. **Curva de aprendizaje:** Si sabes JavaScript, puedes aprender React rápido.
2. **Ecosistema:** Miles de librerías para routing, estado y formularios.
3. **Mercado Laboral:** Es la tecnología frontend más demandada actualmente.
4. **SEO Friendly:** Gracias a soluciones como Next.js.

- **2011:** Creado por Jordan Walke en Facebook para optimizar el News Feed.
- **2012:** Instagram comienza a utilizarlo.
- **2013 (JSConf US):** Se libera como código abierto bajo fuertes críticas iniciales (por mezclar JS y HTML).
- **Actualidad:** Utilizado por Netflix, Airbnb, Twitter y millones más.

Single Page Applications (SPA)

React es el estándar para construir SPAs:

- La página **nunca se recarga** por completo.
- Solo se actualizan los fragmentos de la pantalla necesarios.
- Esto genera una experiencia de usuario similar a la de una aplicación nativa de escritorio o móvil.

El conocimiento de React no se limita a la web:

- **React DOM:** Para navegadores web.
- **React Native:** Para aplicaciones móviles nativas (iOS y Android).
- **React VR/360:** Para experiencias de realidad virtual.

"Aprende una vez, escribe donde sea".

¿Cómo logra React ser tan rápido?

- React crea una **copia ligera** del DOM real en memoria.
- Cuando algo cambia, compara el Virtual DOM con el real.
- Solo aplica los cambios mínimos necesarios en el navegador.

Resultado: Menos consumo de recursos y mayor fluidez.

¿Qué necesitas para empezar?

Antes de entrar al código, debes dominar:

- **JavaScript Moderno (ES6+):** Arrow functions, desestructuración, módulos (import/export).
- **HTML5 y CSS3:** Estructura y diseño básico.
- **Pensamiento en Componentes:** Aprender a dividir un diseño visual en bloques lógicos.

Instalación y Configuración en Windows

Preparando el Entorno

Para desarrollar con React en Windows, necesitamos tres pilares fundamentales:

1. **Entorno de Ejecución:** Node.js y npm.
2. **Editor de Código:** Visual Studio Code.
3. **Terminal:** PowerShell, CMD o Git Bash.

Dato importante

React no se "instala" como un programa de Windows; se gestiona como una dependencia de proyecto.

Paso 1: Node.js y npm

Node.js permite ejecutar JavaScript fuera del navegador. npm es su gestor de paquetes.

- Ve a <https://nodejs.org/>.
- Descarga la versión **LTS** (Long Term Support). Es la más estable.
- Ejecuta el instalador **.msi** y sigue el asistente ("Next, Next, Finish").
- **Tip:** Asegúrate de marcar la casilla "Automatically install the necessary tools".

Paso 2: Verificación

Abre una terminal (PowerShell o CMD) y escribe los siguientes comandos para confirmar que todo está en orden:

```
node -v
```

```
# Debería mostrar algo como: v20.x.x
```

```
npm -v
```

```
# Debería mostrar algo como: 10.x.x
```

Si los comandos no se reconocen, es necesario reiniciar la PC para que se actualicen las variables de entorno (PATH).

Paso 3: Visual Studio Code

Es el editor estándar de la industria para React.

- Descarga desde <https://code.visualstudio.com/>.
- Durante la instalación, marca: **"Add to PATH"** y **"Open with Code"**.
- Esto permitirá abrir proyectos desde la terminal simplemente escribiendo `code ..`

Extensiones de VS Code para React

Para mejorar la productividad en Windows, instala estas extensiones desde el Marketplace:

- **ES7+ React/Redux/React-Native snippets:** Atajos de teclado para crear componentes en segundos.
- **Prettier:** Formatea tu código automáticamente al guardar.
- **ESLint:** Encuentra errores de sintaxis mientras escribes.

¿Vite o Create React App?

Antiguamente se usaba *Create React App* (CRA), pero hoy en día es lento y pesado.

La Recomendación Actual: Vite

- **Vite** es un "build tool" extremadamente rápido.
- Inicia el servidor de desarrollo casi instantáneamente.
- Es el estándar actual recomendado por la comunidad.

Crear el Proyecto

En tu terminal, navega a la carpeta donde quieras guardar tus proyectos y ejecuta:

```
npm create vite@latest mi-primer-app
```

Sigue las instrucciones en pantalla:

1. Selecciona **React**.
2. Selecciona **JavaScript** (o TypeScript si tienes experiencia).

Entrando al Proyecto

Una vez creada la carpeta, debemos instalar las librerías necesarias:

```
cd mi-primer-app  
npm install
```

Esto creará la carpeta `node_modules`, que contiene todo el código de React.

Lanzar la Aplicación

Para ver tu aplicación en funcionamiento:

```
npm run dev
```

Vite te dará una dirección local (ej. **http://localhost:5173**).
Ábrela en Chrome o Edge para ver tu primera aplicación React activa.

Cada vez que quieras programar:

1. Abre la carpeta en **VS Code**.
2. Abre la terminal integrada (**Ctrl** + **ñ**).
3. Ejecuta **npm run dev**.
4. Realiza cambios y observa cómo la página se actualiza sola gracias al **Hot Module Replacement (HMR)**.

Mi Primer Proyecto: Paso a Paso

Al abrir tu proyecto en VS Code, verás:

- **node_modules/**: Todas las librerías de las que depende React.
- **public/**: Archivos estáticos (imágenes, iconos).
- **src/**: Aquí es donde pasaremos el 99% del tiempo.
- **index.html**: El único archivo HTML. Tiene un `<div id="root"></div>`.
- **package.json**: Lista de scripts y dependencias.

Paso 1: El Punto de Entrada

Abre `src/main.jsx`. Es el archivo que conecta React con el HTML real.

```
import React from 'react'  
import ReactDOM from 'react-dom/client'  
import App from './App.jsx'
```

```
ReactDOM.createRoot(document.getElementById('root'))  
  .render(  
    <React.StrictMode>  
      <App />  
    </React.StrictMode>  
  )
```

Nota: `createRoot` le dice a React dónde "dibujarse".

Paso 2: Limpieza para Empezar

Vite crea código de ejemplo. Vamos a limpiar `src/App.jsx` para escribir lo nuestro:

1. Borra todo el contenido de `App.jsx`.
2. Borra el archivo `App.css`.
3. Crea un componente básico desde cero.

Paso 3: Tu Primer Componente

Escribe esto en `src/App.jsx`:

```
function App() {  
  return (  
    <div>  
      <h1>Hola desde React</h1>  
      <p>Este es mi primer componente funcional.</p>  
    </div>  
  );  
}  
  
export default App;
```

Paso 4: Añadiendo un Contador

React brilla con los datos que cambian. Vamos a usar un **Hook**:

```
import { useState } from 'react';

function App() {
  const [cuenta, setCuenta] = useState(0);

  return (
    <div style={{ padding: '20px' }}>
      <h2>Contador: {cuenta}</h2>
      <button onClick={() => setCuenta(cuenta + 1)}>
        Incrementar
      </button>
    </div>
  );
}
```

Paso 5: Ejecución

Si aún no lo has hecho, abre la terminal en VS Code y lanza:

```
npm run dev
```

- Abre la URL que aparece (ej. `http://localhost:5173`).
- Haz clic en el botón.
- **Magia:** React actualiza el número en pantalla automáticamente sin recargar la página.

Paso 6: Pruebas Básicas (DevTools)

Para probar que todo funciona bien:

1. Presiona **F12** o clic derecho → **Inspeccionar**.
2. Ve a la pestaña **Consola**. No debería haber errores en rojo.
3. Si instalaste "React Developer Tools", verás una pestaña llamada **Components** donde puedes ver el valor de "cuenta" en tiempo real.

¿Qué probar si algo falla?

- **Errores de sintaxis:** Vite te mostrará una pantalla roja en el navegador indicando la línea exacta.
- **Importaciones:** Asegúrate de que los nombres de los archivos coincidan (ej. `App.jsx` vs `app.jsx`).
- **Case Sensitivity:** React requiere que los componentes empiecen con **Mayúscula** (`App`, no `app`).

Paso 7: Simular Producción

Para ver cómo se vería tu app terminada antes de subirla a internet:

```
npm run build
```

- Esto crea una carpeta **dist/**.
- Contiene solo HTML, JS y CSS ultra-comprimidos.
- Puedes probar esta versión localmente con: **npm run preview**.

JSX y Componentes Funcionales

¿Qué es JSX?

- Significa **JavaScript XML**.
- Es una extensión de sintaxis que permite escribir HTML dentro de JavaScript.
- No es un string ni es HTML real; se transforma en llamadas a funciones de React (`React.createElement`).

Compilación

Babel o SWC transforman JSX en código JS que el navegador sí puede entender.

Regla 1: Elemento Raíz Único

Un componente debe devolver siempre **un solo elemento** padre.
Correcto

Incorrecto

```
return (  
  <h1>Título</h1>  
  <p>Párrafo</p>  
);
```

```
return (  
  <div>  
    <h1>Título</h1>  
    <p>Párrafo</p>  
  </div>  
);
```

¿Qué pasa si no queremos añadir un `<div>` extra al DOM?

- Usamos **Fragments**: `<> ... </>`.

```
function MyComponent() {  
  return (  
    <>  
      <h1>Sin div extra</h1>  
      <p>El código queda más limpio.</p>  
    </>  
  );  
}
```

Regla 2: Cierre de Etiquetas

A diferencia de HTML5, en JSX **todas** las etiquetas deben cerrarse obligatoriamente.

- `<br />` en lugar de `<br>`
- `` en lugar de ``
- `<hr />` en lugar de `<hr>`

Expresiones en JSX

Para insertar lógica de JavaScript dentro del marcado, usamos llaves `{ }`.

```
function Welcome() {  
  const nombre = "Juan";  
  return (  
    <h1>Hola, {nombre.toUpperCase()}</h1>  
  );  
}
```

- Puedes poner variables, funciones o cualquier expresión válida de JS.
- No puedes poner estructuras de control directas como `if` o `for` dentro de las llaves.

Regla 3: Atributos camelCase

Como JSX es JavaScript, los atributos de HTML usan la convención camelCase:

- `class` → `className`
- `for` → `htmlFor`
- `onclick` → `onClick`
- `tabindex` → `tabIndex`

Estilos como Objetos

Los estilos no se pasan como strings, sino como **objetos** de JS.

```
const miEstilo = {  
  backgroundColor: 'blue',  
  color: 'white',  
  padding: '10px'  
};  
  
function Card() {  
  return <div style={miEstilo}>Contenido</div>;  
}
```

Componentes Funcionales

- Es simplemente una **función de JavaScript**.
- Recibe datos (props) y devuelve elementos de React (JSX).
- Son más fáciles de leer, probar y seguir que las antiguas "Clases".

Importante

El nombre de la función siempre debe empezar con **Mayúscula**.

Estructura Típica

```
// 1. Importaciones
import React from 'react';

// 2. Lógica (JS)
function Perfil() {
  const fecha = new Date().getFullYear();

  // 3. Retorno (UI)
  return <footer>Copyright {fecha}</footer>;
}

// 4. Exportación
export default Perfil;
```

Props (Propiedades)

Las **props** permiten que los componentes sean dinámicos.

```
// Definición del hijo
function Saludo(props) {
  return <h1>Hola, {props.usuario}</h1>;
}
```

```
// Uso en el padre
function App() {
  return <Saludo usuario="Carlos" />;
}
```

Buenas Prácticas: Desestructuración

Es común extraer las propiedades directamente en los parámetros de la función.

```
function Boton({ texto, color }) {  
  return (  
    <button style={{ background: color }}>  
      {texto}  
    </button>  
  );  
}
```

Lógica: Operador Ternario

Para mostrar cosas diferentes según una condición:

```
function Status({ estaLogueado }) {  
  return (  
    <div>  
      {estaLogueado ? <p>Bienvenido</p> : <p>Entrar</p>}  
    </div>  
  );  
}
```

Lógica: Operador Cortocircuito

Si solo quieres mostrar algo cuando la condición es verdadera:

```
function Notificaciones({ mensajes }) {  
  return (  
    <div>  
      <h1>Inbox</h1>  
      {mensajes.length > 0 && <p>Tienes mensajes!</p>}  
    </div>  
  );  
}
```

Listas y la función .map()

Para mostrar múltiples elementos similares:

```
const frutas = ['Manzana', 'Pera', 'Plátano'];

function Lista() {
  return (
    <ul>
      {frutas.map((fruta) => (
        <li key={fruta}>{fruta}</li>
      ))}
    </ul>
  );
}
```

¿Por qué usar Keys?

- Las **keys** ayudan a React a identificar qué elementos han cambiado, se han agregado o se han eliminado.
- Deben ser un identificador único entre hermanos (normalmente un ID de base de datos).
- **Evita:** Usar el índice del array como key si el orden de los elementos puede cambiar.

Props, Estado y Hooks

En React, la información se gestiona de dos formas principales:

1. **Props:** Datos que vienen de fuera (externos).
2. **Estado (State):** Datos que pertenecen al componente (internos).

Regla de Oro

Las **Props** son de solo lectura (inmutables), mientras que el **Estado** es asíncrono y reactivo.

Props: El contrato del componente

Las props permiten que un componente padre configure a sus hijos.

```
function Tarjeta({ titulo, color }) {  
  return <div style={{ background: color }}>  
    {titulo}</div>;  
}
```

```
// El padre controla al hijo  
<Tarjeta titulo="Noticias" color="gray" />
```

Introducción a los Hooks

- Introducidos en React 16.8.
- Son funciones que permiten "engancharse" al estado y al ciclo de vida en componentes funcionales.
- Todos los hooks comienzan con la palabra `use` (ej. `useState`, `useEffect`).

Regla de los Hooks

Solo se pueden llamar en el nivel superior de la función (no dentro de `if`, bucles o funciones anidadas).

Gestionando el Estado: useState

Es el hook más utilizado. Devuelve un array con dos elementos:

1. El valor actual del estado.
2. Una función para actualizarlo.

```
import { useState } from 'react';
```

```
const [valor, setValor] = useState(inicial);
```

Ejemplo: Interruptor (Toggle)

```
function Interruptor() {  
  const [encendido, setEncendido] = useState(false);  
  
  return (  
    <button onClick={() => setEncendido(!encendido)}>  
      {encendido ? "ON" : "OFF"}  
    </button>  
  );  
}
```

Nunca modifiques el estado directamente.

- Incorrecto: `estado = "nuevo"`
- Correcto: `setEstado("nuevo")`

React solo se entera de que debe repintar la pantalla (re-render) si usas la función `set`.

Actualizando Objetos

Al actualizar objetos, debemos mantener el resto de las propiedades usando el operador *spread* (...).

```
const [user, setUser] = useState({ name: 'Ana', age: 25
```

```
// Actualización correcta:  
setUser({ ...user, age: 26 });
```

Sirve para realizar operaciones que están "fuera" del flujo de React:

- Peticiones a APIs (`Fetch`).
- Suscripciones (`WebSockets`).
- Manipulación directa del DOM (raramente).
- Timers (`setInterval`).


```
useEffect(() => {  
  // Código del efecto  
  
  return () => {  
    // Función de limpieza (opcional)  
  };  
}, [dependencias]);
```

El segundo argumento de `useEffect` decide cuándo se ejecuta:

- **Sin array:** Se ejecuta en **cada** renderizado.
- **Array vacío []:** Solo se ejecuta **una vez** (al montar).
- **Con variables [id]:** Se ejecuta cuando **id** cambia.

```
useEffect(() => {  
  fetch('https://api.ejemplo.com/datos')  
    .then(res => res.json())  
    .then(data => setDatos(data));  
}, []); // Se ejecuta al cargar la página
```

Evitando el Prop Drilling

- **Prop Drilling:** Pasar datos por 5 niveles de componentes que no los necesitan.
- **useContext:** Permite crear un "teletransporte" de datos para que cualquier componente hijo acceda a ellos directamente (ej. Tema oscuro, Usuario logueado).

El Hook useRef

Se usa para "tocar" un elemento del HTML directamente o para persistir valores sin disparar un re-render.

```
const inputRef = useRef();

const enfocar = () => {
  inputRef.current.focus();
};

return <input ref={inputRef} />;
```

Puedes crear tus propios hooks para extraer lógica compleja.

- Un custom hook es una función que usa otros hooks dentro.
- Permite compartir lógica entre componentes sin repetir código.
- Ejemplo: `useForm`, `useFetch`, `useAuth`.

Concepto Antiguo	Concepto Moderno (Hooks)
constructor	<code>useState</code> (valor inicial)
<code>componentDidMount</code>	<code>useEffect</code> con <code>[]</code>
<code>componentDidUpdate</code>	<code>useEffect</code> con dependencias
<code>componentWillUnmount</code>	<code>return</code> dentro de <code>useEffect</code>

Navegación con React Router

En una Single Page Application (SPA), no queremos que el navegador recargue la página completa al cambiar de sección.

- **React Router** es la librería estándar para manejar la navegación.
- Permite cambiar la URL y que React renderice solo los componentes asociados a esa ruta.
- Mantiene la sincronización entre la interfaz y la barra de direcciones.

Instalación del Paquete

React Router no viene incluido en el núcleo de React. Debemos instalarlo mediante npm:

```
npm install react-router-dom
```

- **react-router-dom** es la versión específica para aplicaciones web.
- (Existe también **react-router-native** para dispositivos móviles).

Para configurar el enrutamiento, utilizaremos:

1. **BrowserRouter:** El contenedor principal que habilita la navegación.
2. **Routes:** Un contenedor que agrupa todas las rutas posibles.
3. **Route:** Define la relación entre una ruta (*path*) y un componente (*element*).

En tu archivo `App.jsx` o `main.jsx`:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

¡Cuidado!

Nunca uses `<a href="...">` para navegar dentro de tu app React.

- El uso de `<a>` provoca una recarga completa del navegador.
- Se pierde el estado de la aplicación (`useState`).
- Rompe la experiencia de usuario de una SPA.

Componente <Link>

Para navegar internamente, React Router nos ofrece el componente **Link**:

```
import { Link } from 'react-router-dom';

function Navbar() {
  return (
    <nav>
      <Link to="/">Inicio</Link>
      <Link to="/contacto">Contacto</Link>
    </nav>
  );
}
```

Resultado: Cambio de URL instantáneo sin recarga.

Componente <NavLink>

Similar a `Link`, pero añade automáticamente una clase CSS (`.active`) cuando la ruta coincide con la URL actual.

```
<NavLink  
to="/perfil"  
className={({ isActive }) => isActive ? 'active' : ''}  
>  
Mi Perfil  
</NavLink>
```

Es ideal para menús de navegación y barras laterales.

A veces una ruta debe aceptar valores variables, como el ID de un producto.

```
// El ":" indica que "id" es un parámetro variable
<Route path="/usuario/:id"
  element={<DetalleUsuario />} />
```

Esta ruta coincidirá con `/usuario/1`, `/usuario/abc`, etc.

Extrayendo Parámetros

Para leer el valor de la URL dentro del componente, usamos el hook `useParams`:

```
import { useParams } from 'react-router-dom';

function DetalleUsuario() {
  const { id } = useParams();

  return <h1>Viendo el perfil del usuario: {id}</h1>;
}
```

Permite mostrar sub-vistas dentro de una vista principal (ej. un panel de control).

```
<Route path="/admin" element={<AdminLayout />}>  
  <Route path="ventas" element={<Ventas />} />  
  <Route path="usuarios" element={<Usuarios />} />  
</Route>
```

Renderizado de Sub-rutas

En el componente padre (`AdminLayout`), usamos `Outlet` para indicar dónde deben aparecer los hijos.

```
import { Outlet } from 'react-router-dom';

function AdminLayout() {
  return (
    <div className="dashboard">
      <Sidebar />
      <main>
        <Outlet />
        { /* Aquí se renderiza Ventas o Usuarios */ }
      </main>
    </div>
  );
}
```

El Hook useNavigate

A veces necesitamos navegar después de una acción (ej. después de enviar un formulario).

```
import { useNavigate } from 'react-router-dom';

const navigate = useNavigate();

const handleLogin = () => {
  // Lógica de autenticación...
  navigate('/dashboard'); // Redirección automática
};
```

Podemos capturar cualquier URL que no coincida con nuestras rutas usando el asterisco `*`.

```
<Routes>  
  <Route path="/" element={<Home />} />  
  <Route path="*" element={<NotFound />} />  
</Routes>
```

Nota: Siempre debe ser la última ruta definida.

Concepto de Rutas Protegidas

No es una característica nativa, sino un patrón de diseño:

- Crear un componente que verifique si el usuario está logueado.
- Si lo está, devuelve el `<Outlet />`.
- Si no, usa el componente `<Navigate to="/login" />` para redirigir.

Conclusiones de Routing

- React Router transforma una colección de componentes en una **aplicación completa**.
- Provee una experiencia fluida al evitar recargas.
- Permite arquitecturas complejas mediante anidamiento y protección de rutas.
- Es una pieza fundamental en el ecosistema moderno de React.

Custom Hooks y Context API

React permite reutilizar la interfaz (UI) con componentes, pero ¿qué pasa con la lógica?

- A veces, varios componentes necesitan la misma lógica (ej. validar un formulario, detectar la conexión a internet).
- **Custom Hooks** permiten extraer esa lógica a funciones externas.
- **Context API** permite compartir datos entre componentes sin pasar "props" manualmente.

Anatomía de un Custom Hook

- Es una función de JavaScript cuyo nombre comienza con `use`.
- Puede llamar a otros hooks de React (`useState`, `useEffect`, etc.).
- Su objetivo es encapsular lógica compleja para que el componente sea más legible.

Regla de Oro

Un Custom Hook no devuelve JSX; devuelve datos o funciones de utilidad.

Creando useFetch

Imagina un hook para obtener datos de cualquier API:

```
function useFetch(url) {  
  const [data, setData] = useState(null);  
  const [loading, setLoading] = useState(true);  
  
  useEffect(() => {  
    fetch(url)  
      .then(res => res.json())  
      .then(data => {  
        setData(data);  
        setLoading(false);  
      });  
  }, [url]);  
  
  return { data, loading };  
}
```

Consumiendo el Custom Hook

Ahora cualquier componente puede usarlo en una sola línea:

```
function ListaUsuarios() {  
  const { data, loading } = useFetch('/api/users');  
  
  if (loading) return <p>Cargando...</p>;  
  
  return (  
    <ul>  
      {data.map(u => <li key={u.id}>{u.name}</li>)}  
    </ul>  
  );  
}
```

¿Por qué usarlos?

- **DRY (Don't Repeat Yourself):** Evitas duplicar lógica de efectos y estados.
- **Testabilidad:** Puedes probar la lógica del hook de forma independiente a la interfaz.
- **Limpieza:** Los componentes se enfocan en "cómo se ve la app" y no en "cómo funcionan los datos".

El Problema: Prop Drilling

Pasar una prop a través de muchos niveles de componentes (Padre → Hijo → Nieto → Bisnieto) es tedioso y propenso a errores.

- **Context API** crea un "almacén de datos" al que cualquier componente puede suscribirse directamente.
- Es ideal para: Preferencias de tema (oscuro/claro), datos del usuario logueado, idioma (i18n).

Para usar Context necesitamos tres elementos:

1. **createContext:** Crear el almacén.
2. **Provider:** Envolver la aplicación para repartir los datos.
3. **useContext:** El hook para consumir esos datos desde el hijo.

Definición del Contexto

```
import { createContext } from 'react';  
  
// 1. Creamos el contexto  
export const ThemeContext = createContext();
```


Proveyendo los datos

```
function App() {  
  const tema = "oscuro";  
  
  return (  
    <ThemeContext.Provider value={tema}>  
      <MainLayout />  
      { /* Todos los hijos de MainLayout tienen acceso */  
    </ThemeContext.Provider>  
  );  
}
```

Usando useContext

Ahora, en un componente muy profundo:

```
import { useContext } from 'react';
import { ThemeContext } from '../App';

function BotonElegante() {
  const tema = useContext(ThemeContext);

  return (
    <button className={tema}>
      Soy un botón {tema}
    </button>
  );
}
```

Contexto que cambia

Podemos pasar objetos que incluyan funciones para actualizar el estado global:

```
const [user, setUser] = useState(null);

return (
  <UserContext.Provider value={{ user, setUser }}>
    <Login />
  </UserContext.Provider>
);
```

Advertencias sobre Context

Context es potente, pero tiene un costo:

- **Rendimiento:** Cada vez que el valor del Contexto cambia, **todos** los componentes que lo consumen se vuelven a renderizar.
- **Acoplamiento:** Hace que los componentes sean menos reutilizables fuera de ese Contexto.
- **Regla:** No metas todo el estado de la app en Context; úsalo solo para datos verdaderamente globales.

Es común separar la lógica del contexto en su propio componente:

```
export function ThemeProvider({ children }) {  
  const [theme, setTheme] = useState('light');  
  
  return (  
    <ThemeContext.Provider value={{ theme, setTheme }}>  
      {children}  
    </ThemeContext.Provider>  
  );  
}
```

Simplificando el Consumo

Puedes crear un hook específico para tu contexto:

```
export function useTheme() {  
  const context = useContext(ThemeContext);  
  if (!context) throw new Error("Fuera del Provider");  
  return context;  
}
```

```
// En el componente:  
const { theme } = useTheme();
```

- **Custom Hooks** = Reutilización de lógica local.
- **Context API** = Comunicación global.
- Juntos, permiten construir aplicaciones de React limpias, mantenibles y profesionales.